

# Spatial Delta-Rule Memory for Cued Visual Retention: the KDA Accumulator, Its Mathematics, and Why It Works

Visual Attention and Working Memory project (second pass), 2026-09-17

## Abstract

We describe the Kimi Delta Attention (KDA) accumulator as used inside a convolutional encoder in this project, derive its update rule from first principles as a per-location associative memory with learned decay and error-correcting writes, and explain, with equations and shapes, why it solves the cued-orientation retention tasks on which a plain CNN+GRU baseline falls short. On the five-task spatial battery’s orientation family the KDA arm passes a from-scratch gate, learns the cued comparison by curriculum, and holds balanced accuracy 0.994 to 1.000 at retention delays of 4, 12 and 24 blank frames on two seeds, where the plain baseline reaches 0.85 to 0.88. We then set out two analysis programmes for the KDA model and report their first results on a local re-run: an attention analysis that recovers the implicit attention weights and gate maps the recurrence induces (the memory is necessary at every delay; writes fall and retention rises on blanks; the probe attends to the samples and to nothing in between; a linear read of the state recovers the sample angle at  $R^2$  0.99 through 24 blanks), and a psychometric analysis that measures the model as an observer (threshold 2.2 to 2.9 degrees at every delay after training on 15 to 45; no forgetting out to 48 blanks; no distractor interference; sensitivity only to the cue’s contrast and position). The paper is written to teach: every quantity is given with its dimensions, and every claim is tied to a measurement in the repository or marked as a hypothesis.

## 1. Introduction

The project’s benchmark is a five-task battery of  $100 \times 100$  image sequences (orientation change at a cued location with a signed report, spatial binding with a retrocue, cued motion-duration integration, cued motion-change detection after Krauzlis, and image recognition). Each trial gives one label per episode of 4 to 45 frames. In September 2026 an audit established that every task is recoverable from pixels by a fixed observer (experiment 25), that the training recipe used by the previous model lineage collapses any architecture to a constant output within 150 updates (experiment 26), and that a plain per-frame CNN followed by a GRU can learn the cued orientation task at zero delay only by curriculum and cannot hold the cued angle across blank frames without a further curriculum, reaching 0.85 to 0.88 even then (experiments 26 and 27).

Experiment 27 asked one question: does putting a recurrent state *inside* the convolutional stack, at each spatial location and scale, change this? Three update rules were compared against the plain model with everything else identical. Two of them, a convolutional GRU and the spatial KDA accumulator, reached ceiling at every delay on both seeds. This paper is about the second. Section 2 gives the background a reader needs: linear attention as an associative memory, the delta rule, and gated decay. Section 3 gives the module exactly as implemented, with shapes. Section 4 explains why it works on these tasks, including a matrix-form derivation that shows the update is nonexpansive and the write is a projection. Section 5 records the measured results. Sections 6 and 7 describe the attention and psychometric analyses, with first results from a local re-run in 6.7 and 7.4. Section 8 is limitations.

Notation. Bold lower case for vectors, upper case for matrices,  $\sigma$  for the logistic function,  $\odot$  for elementwise product,  $\text{Diag}(\mathbf{a})$  for the diagonal matrix with  $\mathbf{a}$  on the diagonal. A “site” is one spatial position of one feature map at one scale. Frames are indexed by  $t$ .

## 2. Background: three ideas the module combines

### 2.1 Linear attention is an associative memory

Softmax attention computes, for a query  $\mathbf{q}_t$ , a weighted sum of past values with weights  $\propto \exp(\mathbf{q}_t^\top \mathbf{k}_\tau)$ . Linear attention [Katharopoulos et al., 2020] drops the exponential and notices that the sum can then be written with a

running matrix:

$$\mathbf{o}_t = \sum_{\tau \leq t} (\mathbf{q}_t^\top \mathbf{k}_\tau) \mathbf{v}_\tau = \left( \sum_{\tau \leq t} \mathbf{k}_\tau \mathbf{v}_\tau^\top \right)^\top \mathbf{q}_t = S_t^\top \mathbf{q}_t, \quad S_t = S_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top.$$

The matrix  $S_t \in \mathbb{R}^{d_k \times d_v}$  is a *fast weight* [Hinton and Plaut, 1987; Ba et al., 2016; Schlag et al., 2021]: a memory written by Hebbian outer products  $\mathbf{k}\mathbf{v}^\top$  and read by matrix-vector multiplication with a query. If the keys are orthonormal,  $S_t^\top \mathbf{k}_\tau = \mathbf{v}_\tau$  exactly; the memory has  $d_k$  independent slots. If keys are not orthogonal, reads are contaminated by cross-talk. This is the mechanism we want at each image location: store “what was here” under a key that says “which kind of thing,” and read it back later.

## 2.2 The delta rule replaces instead of accumulates

Plain Hebbian accumulation has a defect: writing the same key twice adds the values. The delta rule [Widrow and Hoff, 1960], used as the update of DeltaNet [Schlag et al., 2021; Yang et al., 2024], first computes what the memory currently predicts for the key, then writes only the *error*:

$$\mathbf{e}_t = \mathbf{v}_t - S_{t-1}^\top \mathbf{k}_t, \quad S_t = S_{t-1} + \beta_t \mathbf{k}_t \mathbf{e}_t^\top.$$

With  $\beta_t = 1$  and  $\|\mathbf{k}_t\| = 1$  this makes the memory return exactly  $\mathbf{v}_t$  for key  $\mathbf{k}_t$  afterwards, overwriting whatever was stored under that key without touching orthogonal keys. It is one step of gradient descent on  $\frac{1}{2} \|\mathbf{v}_t - S^\top \mathbf{k}_t\|^2$  with step size  $\beta_t$ , which is why it is called an error-correcting or “test-time learning” update [Sun et al., 2024].

## 2.3 Gated decay makes the memory forget on purpose

Recurrent state that never decays saturates. Gated linear attention [Yang et al., 2023], Mamba-2 [Dao and Gu, 2024] and Gated DeltaNet [Yang et al., 2024] multiply the old state by an input-dependent factor in  $(0, 1)$  before writing. Kimi Delta Attention [Kimi Linear, 2025, eq. 1] uses a *channel-wise* decay, one factor per key dimension, and combines it with the delta rule:

$$\bar{S}_t = \text{Diag}(\alpha_t) S_{t-1}, \quad \mathbf{e}_t = \mathbf{v}_t - \bar{S}_t^\top \mathbf{k}_t, \quad S_t = \bar{S}_t + \beta_t \mathbf{k}_t \mathbf{e}_t^\top, \quad \mathbf{o}_t = S_t^\top \mathbf{q}_t.$$

$\alpha_t \in (0, 1)^{d_k}$  is the retention per key dimension and  $\beta_t \in (0, 1)$  the write strength; both are computed from the current input. Decay first, then correct, then read after writing. That single line is the whole module; everything else is plumbing that decides where the vectors come from and where the output goes.

## 3. The spatial KDA accumulator as implemented

The project’s implementation is `PreAttentiveVision/TemporalIntegration/accumulators.py:SpatialKDA` and `kda_update`, placed inside the conv stack by `WorkingMemory/PlainBaseline/accum.py`. Nothing below is a proposal; it is what ran in experiment 27.

### 3.1 Where the module sits

The encoder is the plain baseline’s four convolution blocks (stride 2 each, GroupNorm, ReLU), applied to three stacked consecutive frames as 9 input channels, giving maps of  $32 \times 50 \times 50$ ,  $64 \times 25 \times 25$ ,  $96 \times 13 \times 13$  and  $128 \times 7 \times 7$ . After each of the three coarser blocks, a  $1 \times 1$  convolution projects the block output  $H_t^s$  to a 32-channel map  $U_t^s$ , the accumulator updates its state and emits a 32-channel field  $O_t^s$ , and the next block receives the concatenation  $[H_t^s, O_t^s]$ . The deepest concatenation,  $160 \times 7 \times 7$ , is flattened to a 256-d feature through a linear layer and ReLU, a single GRU with 256 hidden units reads that feature per frame, and a linear head per task reads the GRU’s final state. The plain baseline is the same network with the accumulators removed, so the only difference between the two is the spatial state.

$$U_t^s = P_s H_t^s \text{ (32 ch)}, \quad (O_t^s, S_t^s) = \text{KDA}_s(U_t^s, S_{t-1}^s), \quad H_t^{s+1} = \text{Block}_{s+1}([H_t^s, O_t^s]).$$

## 3.2 What the module computes at one site

A  $3 \times 3$  convolution maps the 32-channel input map to 82 channels, which are split into two heads of 41:  $\mathbf{q}$  (8),  $\mathbf{k}$  (8),  $\mathbf{v}$  (16),  $\alpha$  logits (8) and a  $\beta$  logit (1). Then, at every site independently but with shared weights,

$$\mathbf{q}_t = \frac{\tilde{\mathbf{q}}_t}{\|\tilde{\mathbf{q}}_t\|}, \quad \mathbf{k}_t = \frac{\tilde{\mathbf{k}}_t}{\|\tilde{\mathbf{k}}_t\|}, \quad \alpha_t = \sigma(\tilde{\alpha}_t) \in (0, 1)^8, \quad \beta_t = \sigma(\tilde{\beta}_t) \in (0, 1),$$

followed by the KDA update of Section 2.3 with  $S_t \in \mathbb{R}^{8 \times 16}$  per head. The two 16-d outputs are concatenated and passed through a learned  $1 \times 1$  convolution  $32 \rightarrow 32$  to give  $O_t$ . The state is initialised to zero at the first frame of every episode and is never shared across episodes.

Initialisation: the gate weights start at zero and the biases at  $\log 9$  for  $\alpha$  and 0 for  $\beta$ , so at initialisation every site retains 90% of its state per frame and writes with strength 0.5, independently of the input. Everything, including the gate weights, is trained.

## 3.3 Sizes

| Quantity                                     | Value                                               |
|----------------------------------------------|-----------------------------------------------------|
| Sites with state                             | $625 + 169 + 49 = 843$ (three scales)               |
| State per site                               | $2 \text{ heads} \times 8 \times 16 = 256$ floats   |
| State per example                            | 215,808 floats (0.82 MiB fp32)                      |
| Parameters of the kda arm                    | 2,747,240 (plain: 2,197,746; convgru: 2,839,154)    |
| Cost of the final delay stage on an RTX 3090 | 716 s (plain 489 s, convgru 851 s, opponent 1334 s) |

The state is eight times larger than the ConvGRU’s 32 floats per site, but the accumulator itself has few parameters: the  $3 \times 3$  input convolution ( $32 \times 82 \times 9 + 82$ ) and the  $1 \times 1$  output ( $32 \times 32 + 32$ ), about 24.7k per scale.

## 4. Why it works on these tasks

### 4.1 The task’s demand, stated as a computation

At D0 the orientation task is a three-way interaction: the sign glyph says which location and which direction to report, and the answer is the sign of the rotation between the sample frames and the probe. With three frames stacked as input channels, the rotation at any location is a first-order feature of the first convolution, so the D0 task is a matter of routing and sign, which curriculum from a two-way rung solved for every arm (experiments 26, 27). The delays are different. At D4 the probe’s three-frame window contains two blanks and the probe; the sample angle is no longer in the input. Something has to hold, for each location, “the Gabor here had orientation  $\theta$ ,” through 4, 12 or 24 frames of gray that carry no information, and then let the probe frame be compared with it.

The plain baseline can only do this in its 256-d GRU after the flatten. It has to compress four locations’ orientations plus the cue into a vector and protect that vector through the blanks with gates that see only the flattened feature. It got there partially (0.85 to 0.88) and only after two extra ladder stages.

### 4.2 A per-site memory with keys is the right shape

The KDA state lives at the site. At the  $25 \times 25$  scale a Gabor of radius 12 px covers a few sites, and each of them can store the local orientation feature  $\mathbf{v}$  under a key  $\mathbf{k}$  that the  $3 \times 3$  input convolution computes from the *appearance* of the input, not its position. The four Gabors do not compete for capacity because they are at different sites. This alone removes the bottleneck that hurts the plain model.

The key mechanism is what happens during blanks. A blank frame produces some input  $U$  at every site, hence some  $\mathbf{k}_{\text{blank}}, \mathbf{v}_{\text{blank}}$ . If the network simply wrote them, the delta rule would replace the stored value with the blank’s value after one frame. Two things prevent this, and both are learnable from the data:

1. **Write gating.**  $\beta_t$  is a function of the input. The network can learn  $\beta \approx 0$  on blanks and  $\beta \approx 1$  on stimulus frames. Then blanks do not write at all.
2. **Key addressing.** Even with  $\beta > 0$ , a write under a key orthogonal to the stored key does not disturb the stored value. In an 8-d key space a blank can be assigned its own direction. Reads at the probe use the query the probe generates, which the network can align with the stimulus key.

Retention itself is  $\alpha$ . A per-dimension retention of 0.99 over 24 frames keeps 79% of the stored signal; 0.9 keeps 8%. The gate starts at 0.9 and has to learn to open toward 1 on the dimensions that hold the stimulus, which the ladder (D1, D2, then D4, then D12/D24) asks for in steps. On the pod the kda arm was already at 1.000 on D2 in the first ladder stage and needed no recovery afterwards, which is what one expects if the gate has to move only a little to cover each new delay.

### 4.3 The update is a projection and is nonexpansive

Write the update in matrix form. Substituting  $\mathbf{e}_t$  into the write,

$$S_t = \bar{S}_t + \beta_t \mathbf{k}_t (\mathbf{v}_t - \bar{S}_t^\top \mathbf{k}_t)^\top = (I - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \text{Diag}(\alpha_t) S_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top.$$

Because  $\|\mathbf{k}_t\| = 1$ , the matrix  $I - \beta_t \mathbf{k}_t \mathbf{k}_t^\top$  has eigenvalue  $1 - \beta_t$  along  $\mathbf{k}_t$  and 1 on the seven orthogonal directions; with  $\beta_t \in (0, 1)$  it is a partial projection that shrinks the state only along the key being written.  $\text{Diag}(\alpha_t)$  has entries in  $(0, 1)$ . The transition matrix  $A_t = (I - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \text{Diag}(\alpha_t)$  therefore has spectral norm at most 1: the old state is never amplified, so 24 or 100 blank frames cannot blow the state up, and gradients through the state are likewise bounded by products of norms at most 1. (This is a statement about the linear transition applied to the old state; it does not bound the values written, and it is not a convergence proof for training.)

The read after the write gives, for a query equal to the key just written,

$$S_t^\top \mathbf{k}_t = (1 - \beta_t) \bar{S}_t^\top \mathbf{k}_t + \beta_t \mathbf{v}_t,$$

a convex combination of what the memory already believed for that key and the new value.  $\beta$  is therefore literally a learning rate for the memory, and  $1 - \beta$  a “trust the past” weight.

### 4.4 Unrolling gives implicit attention

Iterating the matrix form from  $S_0 = 0$ ,

$$S_t = \sum_{\tau \leq t} \left( \prod_{s=\tau+1}^t A_s \right) \beta_\tau \mathbf{k}_\tau \mathbf{v}_\tau^\top, \quad \mathbf{o}_t = S_t^\top \mathbf{q}_t = \sum_{\tau \leq t} w_{t,\tau} \mathbf{v}_\tau, \quad w_{t,\tau} = \beta_\tau \mathbf{q}_t^\top \left( \prod_{s=\tau+1}^t A_s \right) \mathbf{k}_\tau.$$

The output at frame  $t$  is a weighted sum of the values written at every earlier frame, with a scalar weight  $w_{t,\tau}$  that depends on the query, the key written at  $\tau$ , the write strength at  $\tau$ , and every decay and every interfering write in between. These weights are the module’s attention over the past. They are not normalised, can be negative, and are exact: computing them costs one  $8 \times 8$  matrix product per intervening frame per site. Section 6 builds the attention analysis on this identity.

### 4.5 What the comparison arms tell us

The ConvGRU arm, with 32 floats per site and a learned  $3 \times 3$  recurrent kernel, reached the same ceiling. The opponent-trace arm, whose state is two leaky averages with learned but input-*independent* retention and no write gate, reached ceiling through D12 and split at D24 (0.55 and 1.00 on two seeds). The common ingredient of the two arms that worked is input-dependent gating of retention and writing at each site. KDA adds key-addressed storage and error-correcting writes on top; whether those add anything beyond gating is an open ablation (Section 8). Both gated arms cost nothing at D0 and passed the from-scratch gate, so the state is not in the way when it is not needed.

## 4.6 A biological reading, kept modest

The project’s scaffold treats activated long-term memory as synaptic weights. The KDA state is a per-location matrix of fast weights written by an outer product and decaying with a learned time constant, which is the shape of a synaptic short-term memory [Mongillo et al., 2008]: information held in facilitated synapses rather than in persistent firing.  $\alpha$  plays the role of a facilitation decay,  $\beta$  of a plasticity gate that could be neuromodulatory, and the query read plays the role of a probe stimulus reactivating what the synapses hold. This is an analogy that motivates the analyses in Section 6, not a claim that the module is a cortical circuit.

## 5. Measured results (experiment 27)

All arms: everything trainable from scratch, one learning rate ( $10^{-4}$ , Adam), batch 64, no clipping, input centred, three stacked frames. Program per arm and seed: ring rung from scratch (gate), real task at D0 from the ring model, then a delay ladder D0/1/2, D0/2/4, D0/4/12/24. Test balanced accuracy on 512 trials at the fixed test seed, terminal checkpoint of each stage.

| Arm                                | Gate (ring) | D0<br>(curriculum) | D2 in first<br>ladder stage | D4          | D12         | D24         |
|------------------------------------|-------------|--------------------|-----------------------------|-------------|-------------|-------------|
| plain, seeds<br>1/2                | 1.000/1.000 | 0.998/0.986        | 0.523/0.580                 | 0.867/0.875 | 0.867/0.883 | 0.848/0.875 |
| convgru                            | 1.000/1.000 | 1.000/1.000        | 1.000/1.000                 | 1.000/1.000 | 1.000/1.000 | 1.000/1.000 |
| opponent<br>(learned<br>retention) | 1.000/1.000 | 0.990/0.996        | 0.977/0.990                 | 1.000/1.000 | 0.992/0.990 | 0.551/0.996 |
| <b>kda</b>                         | 1.000/0.998 | 1.000/0.998        | 1.000/1.000                 | 1.000/1.000 | 1.000/1.000 | 1.000/1.000 |

Source: WorkingMemory/PlainBaseline/runs/cloud\_20260917\_010358/final\_tables.md and the per-stage receipts. Terminal weights of these runs were not retrieved from the pod; a local re-run of the kda arm with checkpoints kept is in progress (runs/local\_kda\_program\_20260917/).

## 6. Attention analysis for the KDA model

The goal is to see what the memory attends to, when it writes, and what it holds, at the level of sites and frames, and to test each observation by intervention. Every quantity below is computed from a trained model on held-out trials at the test seed, per task condition, and reported per cued and uncued location.

### 6.1 Gate maps

Record  $\alpha_t$  (8 per head) and  $\beta_t$  per site, scale and frame. Plot, per frame, the spatial map of  $\beta$  and of  $\bar{\alpha} = \text{mean}(\alpha)$ . Predictions if the memory works as Section 4 says:  $\beta$  high on sample frames and near zero on blanks;  $\bar{\alpha}$  near 1 on blanks at the Gabor sites; at the probe,  $\beta$  may rise again (the probe is written) or stay low (the probe is compared without writing). Quantify with the mean  $\beta$  and  $\bar{\alpha}$  on stimulus versus blank frames at Gabor versus background sites, with 95% bootstrap intervals over trials.

### 6.2 Implicit attention weights

Compute  $w_{t,\tau}$  from the unrolled identity (Section 4.4) for the probe frame  $t$  at every site: which earlier frames does the probe’s query read from? Report the attention profile over  $\tau$  averaged over sites within the cued Gabor, the uncued Gabors and the background, for each delay. The signature of retention is a profile that peaks on the sample frames and is near zero on blanks even at D24. The signature of the failure mode (overwriting) is mass on the last few blanks.

### 6.3 State probes

Freeze the model, collect  $S_t^s$  at each site, and fit linear decoders from the state to (a) the sample orientation at that site (regression on  $\cos 2\theta, \sin 2\theta$ ), (b) whether the site is cued, and (c) the eventual label. Decode at every frame to get a time course: does orientation information in the state stay constant across the blanks, decay, or get re-encoded at the probe? Use the project’s existing probing pattern (AttentionContextComparator/V2/preflight\_probe.py) with the fields replaced by KDA states, and report cross-validated  $R^2$  or balanced accuracy per frame and per scale.

### 6.4 Interventions

Each intervention is applied at evaluation only and scored against the undisturbed model on the same trials:

- *State reset* before the probe: removes the memory; accuracy should fall to chance at every delay. This is the necessity check.
- *Write clamp on blanks* ( $\beta \leftarrow 0$  during blanks): if accuracy is unchanged, blanks are not being written; if it rises, the model was leaking.
- *Retention clamp* ( $\alpha \leftarrow 1$  during blanks): tests whether the learned decay is doing anything beyond what perfect retention would do.
- *Site swap*: exchange the states of the cued and one uncued Gabor site group before the probe. Correct behaviour follows the swapped memory, which shows the memory is site-local and the readout is cue-directed.
- *Key ablation*: project the state onto the top- $m$  key directions by singular value and re-run; the number of directions needed for full accuracy measures how many slots the memory uses.

### 6.5 Readout attribution

The final answer is read by the GRU and head from the flattened deepest map. Compute the gradient of the correct logit with respect to  $O_t^s$  at each site and frame, and the input-times-gradient map. Together with 6.2 this separates “what the memory holds” from “what the readout uses.”

### 6.6 Output

A per-model report with: gate-map figures at D0/4/12/24; attention profiles; probe time courses; an intervention table; and a one-paragraph verdict per prediction in 4.2. The same code runs on the ConvGRU arm (gate maps and interventions only, since it has no key structure), which gives the comparison that the ablation in Section 8 needs.

### 6.7 First results (local KDA re-run, seed 1, final delay model)

`kda_probe.py` was run on runs/local\_kda\_program\_20260917/kda\_s1/delayC/terminal.pt (test BA 1.000 at D0/4/12/24, the same as the pod run) with 128 trials per delay at the psychometric seed. Numbers are for the 25×25 scale unless stated; the two coarser scales behave the same way with weaker probes.

*Interventions.* Resetting the state before the probe gives BA 0.48, 0.52, 0.52 and 0.52 at D0, D4, D12 and D24: the memory is necessary at every delay, including D0, where the three-frame stack would in principle allow a direct comparison. Clamping  $\beta = 0$  on every blank, or  $\alpha = 1$  on every blank, leaves BA at 1.000 at all delays: nothing written during blanks is used, and the learned retention is already as good as perfect retention.

*Gates.* Mean write strength  $\beta$  at Gabor sites is 0.77 on sample frames, 0.52 on blanks (0.58 at D4, 0.52 at D24) and 0.75 at the probe; on background sites it is 0.54 throughout. Mean retention  $\bar{\alpha}$  at Gabor sites is 0.94 on stimulus frames and 0.96 on blanks. So the network learned to write less and retain more when the input is gray, as Section 4.2 predicted, and it did so with no site-specific tuning at initialisation. The gates at cued and uncued Gabor sites are identical to two decimals: the memory writes every Gabor, and the cue is applied downstream by the readout, not by the write gate.

*Implicit attention.* The probe frame’s attention weights  $|w_{t,\tau}|$  over the past, averaged over the cued Gabor sites, peak on the second sample frame (0.22 at D0, 0.18 at D4, 0.16 at D12, 0.13 at D24) and on the probe itself (0.13), and are 0.002 to 0.004 on every blank frame at D24. The memory’s read at the probe is a read of the samples, not of the recent gray.

*State probe.* A linear regression from the state at the cued site to  $(\cos 2\theta, \sin 2\theta)$  of the sample orientation, fit on half the trials and scored on the other half, gives  $R^2 = 0.99$  on every frame from the first sample through the 24th blank at the  $25 \times 25$  scale (0.96 at  $13 \times 13$ , about 0.75 at  $7 \times 7$ ). The angle is held losslessly, at the resolution of this probe, through the whole delay, and it is held at the finest scale.

Together these say the module works as the derivation claimed: a site-local, key-addressed store written on stimulus frames, protected during blanks by high retention and a lowered write gate, and read at the probe by a query that lands on the sample entries. What it does not say is how the readout selects the cued site, which is the next probe (readout attribution, 6.5, and site swap, 6.4, were not run here).

## 7. Psychometric analysis for the KDA model

Psychophysics treats the model as an observer and measures how performance depends on stimulus strength, delay and distraction. The generator already exposes the needed parameters; the plan is to sweep them on held-out trials, fit standard psychometric functions, and compare arms.

### 7.1 Independent variables and the functions to fit

1. **Rotation magnitude** (signal strength):  $2.5^\circ, 5^\circ, 7.5^\circ, 10^\circ, 15^\circ, 22.5^\circ, 30^\circ, 45^\circ$ , at D0 and at each delay. Fit a cumulative Gaussian with lapse,  $\psi(x) = \gamma + (1 - \gamma - \lambda) \Phi((x - \mu)/s)$  with guess rate  $\gamma = 0.5$ , by maximum likelihood [Wichmann and Hill, 2001], and report threshold (75% point), slope  $1/s$  and lapse  $\lambda$  with bootstrap intervals. The training set used  $15/30/45^\circ$ , so  $2.5^\circ$  to  $10^\circ$  are extrapolations that measure the observer’s precision, not its training.
2. **Delay** (forgetting function): 0, 1, 2, 4, 8, 12, 16, 24, 32, 48 blanks at fixed magnitude  $15^\circ$ . Fit an exponential  $a + (b - a)e^{-d/\tau}$  and report the time constant  $\tau$  and the asymptote; a model with a genuine memory shows a flat function out to and beyond the trained 24. Beyond 24 is an extrapolation and the interesting part.
3. **Cue reliability**: cue contrast scaled 1.0, 0.5, 0.25, 0.1; glyph position jitter 0, 2, 4 px. Measures how much of the accuracy depends on the cue being exactly as trained.
4. **Distraction**: number of uncued Gabors 0, 1, 2, 3 and a variant where uncued Gabors rotate by the same magnitude as the target (maximal interference). Retention that is site-local should be insensitive to this; a shared-vector memory should not.
5. **Set size and load** (for the other families later): number of study items in recognition, number of patches in motion.

### 7.2 Protocol

Per point: 512 fresh trials from a dedicated psychometric stream (a seed distinct from train, validation and test), balanced labels and locations, fixed argmax, balanced accuracy and AUC, 2,000-replicate bootstrap intervals over trials. Same trials for every model (paired comparison). Report per condition, never pooled. The magnitude sweep needs a generator option for the magnitude set; the delay sweep needs none; the cue and distraction sweeps need two small options in the variant generator. All three are additions to `WorkingMemory/PlainBaseline/variants.py`, and the sweep runner is `WorkingMemory/PlainBaseline/analysis/psychometric.py`.

### 7.3 What the curves would tell us

- A threshold well below  $15^\circ$  at D0 that rises with delay tells us the memory stores the angle with finite precision that degrades; a threshold that does not rise says retention is lossless at the tested resolution.
- A forgetting time constant far beyond 24 frames says the gates learned near-perfect retention, not a decay tuned to the training delays.
- Insensitivity to distraction and sensitivity to cue contrast would say the bottleneck is cue detection, not memory, which redirects the next architectural question toward attention.
- Comparing KDA with ConvGRU and plain on the same curves turns “1.000 versus 0.87” into a description of *how* they differ: precision, capacity or persistence.

## 7.4 First results (local KDA re-run, seed 1, final delay model)

`psychometric.py` was run on the same checkpoint as 6.7, 512 trials per point at the psychometric seed, balanced accuracy with 95% trial-bootstrap intervals (about  $\pm 0.02$  at 0.9,  $\pm 0.04$  at 0.75).

*Magnitude.* Trained only on 15°, 30° and 45°, the model reads much smaller rotations:

| delay | 2.5°  | 5°    | 7.5°  | 10°   | 15°   | threshold (75%) |
|-------|-------|-------|-------|-------|-------|-----------------|
| D0    | 0.707 | 0.896 | 0.967 | 0.992 | 1.000 | 2.9°            |
| D4    | 0.750 | 0.963 | 1.000 | 1.000 | 1.000 | 2.5°            |
| D12   | 0.762 | 0.953 | 1.000 | 1.000 | 1.000 | 2.4°            |
| D24   | 0.779 | 0.947 | 0.996 | 1.000 | 1.000 | 2.2°            |

The threshold does not rise with delay; if anything it is lowest at D24. Whatever precision the memory stores at the samples, it keeps through 24 blanks. The fitted lapse is 0.000 at every delay.

*Forgetting function.* At 15°, BA is 1.000 at delays 0, 1, 2, 4, 8, 12, 16, 24, 32 and 48. The exponential fit is degenerate (asymptote 1.000). Trained to 24, the memory holds at 48; the learned retention is not tuned to the training delays.

*Cue reliability (D4).* BA falls from 1.000 at the trained glyph to 0.947 and 0.850 with 2 px and 4 px position jitter, to 0.895 at half contrast, 0.537 at quarter contrast, and 0.500 at a tenth. Jitter and contrast loss compound (0.602 at half contrast with 4 px jitter). This is the one variable the observer is sensitive to.

*Distraction (D12).* BA is 0.994 with no uncued Gabors and 1.000 with one, two or three, whether they rotate as in the real task or all by the target’s magnitude. The memory does not suffer interference from the other locations.

*Reading.* The three memory-side measurements (precision, persistence, distraction) all say the same thing as the probes in 6.7: the stored representation is precise, held without loss and site-local. The only fragility is the cue. Where the plain model’s limit was retention, this model’s limit is cue detection and the readout’s use of it, which moves the next architectural question from memory to attention: how the cue selects a site and how robust that selection is to cue appearance. That question is the one the project’s attention components were built for, and it can now be asked against a baseline whose memory is not the bottleneck.

## 7.5 Relation to human data

The battery’s parameters were chosen to be human-plausible, and the Krauzlis task follows a published paradigm. Where human thresholds exist for orientation change detection and delayed orientation matching, the model’s thresholds and forgetting function can be placed beside them. This is a comparison of shapes, not a claim of fit; the model has no reaction time and no lapse from inattention other than what its lapse parameter absorbs.

## 8. Limitations and the next measurements

- Only the orientation family has been run. The same program on motion, binding, recognition and Krauzlis is the first next step, with ladder rungs built per family.
- Two seeds. The opponent arm’s split at D24 shows seed variance is real; the kda arm’s ceiling on two seeds is strong but not a population claim.
- The kda and convgru arms tie. An ablation is needed to separate gating from key-addressed storage: a per-site gated scalar memory (ConvGRU with 256 units), and KDA with  $\beta$  fixed to 1 or with the delta term removed (plain Hebbian write).
- Delays were reached by a curriculum. Whether the kda arm learns D24 directly from the D0 model, without the D1/D2 and D2/D4 stages, is untested and matters for the claim that the memory is easy to learn. (The psychometric results in 7.4 show that once learned the memory generalises to delays and magnitudes far outside training; they say nothing about how it was learned.)
- The pod runs’ weights were lost; the state analysis in 6.7 is from a local re-run (seed 1) that reproduced the pod result. Site-swap and readout-attribution probes have not been run.
- The stack of three frames gives the first convolution direct access to adjacent-frame change. A per-frame version of the same encoder would show whether the accumulator can also do the D0 comparison on its own.

## References

Locators are given as recorded in the repository’s research notes where available; those marked “verify” are from memory and must be checked before citation in any submission.

- Ba, J., Hinton, G., Mnih, V., Leibo, J. Z., Ionescu, C. (2016). Using fast weights to attend to the recent past. NeurIPS. arXiv:1610.06258 (verify).
- Ballas, N., Yao, L., Pal, C., Courville, A. (2016). Delving deeper into convolutional networks for learning video representations. ICLR. arXiv:1511.06432.
- Dao, T., Gu, A. (2024). Transformers are SSMS: generalized models and efficient algorithms through structured state space duality (Mamba-2). ICML. arXiv:2405.21060 (verify).
- Hinton, G. E., Plaut, D. C. (1987). Using fast weights to deblur old memories. Proc. Cognitive Science Society (verify).
- Katharopoulos, A., Vyas, A., Pappas, N., Fleuret, F. (2020). Transformers are RNNs: fast autoregressive transformers with linear attention. ICML. arXiv:2006.16236 (verify).
- Kimi Team (2025). Kimi Linear: an expressive, efficient attention architecture. arXiv:2510.26692, equation 1; implementation [github.com/MoonshotAI/Kimi-Linear](https://github.com/MoonshotAI/Kimi-Linear).
- Mongillo, G., Barak, O., Tsodyks, M. (2008). Synaptic theory of working memory. Science 319:1543–1546 (verify).
- Schlag, I., Irie, K., Schmidhuber, J. (2021). Linear transformers are secretly fast weight programmers. ICML. arXiv:2102.11174 (verify).
- Sun, Y. et al. (2024). Learning to (learn at test time): RNNs with expressive hidden states. arXiv:2407.04620 (verify).
- Wichmann, F. A., Hill, N. J. (2001). The psychometric function: I. Fitting, sampling, and goodness of fit. Perception & Psychophysics 63:1293–1313 (verify).
- Widrow, B., Hoff, M. E. (1960). Adaptive switching circuits. IRE WESCON Convention Record (verify).
- Yang, S., Wang, B., Shen, Y., Panda, R., Kim, Y. (2023). Gated linear attention transformers with hardware-efficient training. arXiv:2312.06635 (verify).
- Yang, S., Kautz, J., Hatamizadeh, A. (2024). Gated Delta Networks: improving Mamba2 with delta rule. ICLR 2025. arXiv:2412.06464.
- Project documents: experiments 25, 26, 27 in [LabJournal/experiments/](#); [PreAttentiveVision/TemporalIntegration/README.md](#) (original three-accumulator specification); [WorkingMemory/PlainBaseline/accum.py](#).